

Comprendre la réflexivité dans Pharo

Larisa Safina
META

Pharo

- Un langage de programmation dynamique et purement orienté objet.
- Basé sur Smalltalk, il offre un environnement de programmation en direct.
- Utilisé dans la recherche, l'éducation et le prototypage rapide.

La réflexivité

Permet d'inspecter et de modifier sa propre structure et son comportement à l'exécution.

- Pharo peut analyser et modifier ses classes, méthodes et objets dynamiquement.
- Essentiel pour la métaprogrammation, le débogage et la modification dynamique du système.

Avantages de la Réflexivité

- Flexibilité : Modifier le comportement sans recompilation.
- Interactivité : Retour immédiat et modifications en direct.
- Puissance de débogage : Inspection et modification dynamique.
- Extensibilité : Adapter le langage à des besoins spécifiques.

La réflexivité dans Pharo

- Introspection : examiner les objets, classes et méthodes.
- Métaprogrammation :
 - manipuler des structures de code.
 - modifier le comportement dynamiquement à l'exécution.
- Live Coding : modifier le code pendant l'exécution du système.

Inspection de la structure

Object methods.

Object selectors.

Object class classVarNames.

Object class subclasses.

Object methods last temporaryVariables

RPackageOrganizer default packages

...

Inspection de la structure

Inspector on RBPackge

a RBPackge class (RBPackge)

Raw Definition All References Comment Breakpoints **Meta**

Class hierarchy

- ▼ ProtoObject
 - ▼ Object
 - ▼ Behavior
 - ▼ ClassDescription
 - ▼ Class
 - ▼ ProtoObject class
 - ▼ Object class
 - ▼ RBEntity class
 - RBPackge class

allClassVarNames

allInstVarNames

allInstances

Filter...

```
1 allClassVarNames
2 "Answer the names of the receiver's and the receiver's ancestor's
3 class variables."
```

Métaprogrammation:

Installer une classe et une méthode

```
Object subclass: #Greeter  
  instanceVariableNames: ''  
  classVariableNames: ''  
  poolDictionaries: ''  
  category: 'Examples'.
```

```
Greeter compile: 'sayHello  
^''Bonjour !'''.  
Greeter new sayHello
```

Disponible
immédiatement
sans redémarrage.

Exécuter une méthode par nom

```
Greeter compile: 'sayHello  
^'' 'Bonjour !'' '.
```

```
Greeter new perform: #sayHello
```

Pour exécuter avec des arguments:

```
Greeter new perform:  
#myMethodWithArg with: #anArgument.
```

Exécuter une méthode avec une méthode compilée

```
| compiledMethod |
```

```
compiledMethod :=  
  Person methods detect: [:method |  
    method selector = 'age'].
```

```
Person new executeMethod:  
compiledMethod.
```

Ajouter un package

```
(RPackageOrganizer default  
  includesPackageNamed: packageName)  
  ifTrue: [ RPackageOrganizer default  
    removePackageNamed: packageName  
  ].
```

```
RPackageOrganizer default  
  addPackageNamed: packageName.
```

Autre façon d'installer la classe

```
Object classInstaller make: [ :builder |  
    builder  
        superclass: Object;  
        name: (confClass at: 'className');  
        slotsFromString: '';  
        sharedVariablesFromString: '';  
        package: 'packageName';  
        environment: Object environment  
    ].
```

Live Coding et Débogage

- Le débogueur de Pharo permet de modifier et de réexécuter le code à la volée.
- Les modifications prennent effet instantanément.
- Facilite le prototypage rapide et la résolution de problèmes.

Live Coding

The screenshot shows a live coding environment with a project tree on the left containing folders like OSWindow-Tests, Ombu, Ombu-Tests, OpalCompiler-Core, OpalCompiler-Tests, ParametrizedTests, PerformMessageCommandLi, and Person. The 'Person' folder is selected. The main editor displays the 'Person' class with the 'instance side' tab active. The code in the editor is:

```
initialize
  age := 20
```

A dialog box is open in the center, titled "Unknown variable: age please correct, or cancel:". It contains three options: "Declare new temporary variable", "Declare new instance variable", and "Cancel". The "Declare new instance variable" option is selected. The dialog also has a text input field and a "Cancel" button.

At the bottom of the editor, there is a toolbar with various icons and tabs. The tabs include "Dependencies", "Person", "Comment", and two instances of "+ *Inst. side meth x". The "Person" tab is active.

Débogage

The screenshot shows the Opal Playground interface. At the top, the title bar says "Playground". Below it is a toolbar with icons for "Do it", "Publish", "Bindings", "Versions", and "Pages". The main code area contains a single line of code: `1 Person new age`. Below the code area, a message box displays the error: "Instance of Person did not understand #age". Underneath the error message is a "Stack" section with a table of stack frames.

Class	Method	Package
UndefinedObject	Dolt	-
OCReceiverDoltSemanticScope (OCDoltSemanticScope)	evaluateDolt:	OpalCompiler-
OpalCompiler	evaluate	OpalCompiler-

Below the stack table is another toolbar with buttons: "Proceed", "Info", "Over", "Through", "Run to", "Restart", "Return", "Where is?", "Create" (highlighted with a red circle), and "Advanced Step". At the bottom, the code area shows the same line: `1 Person new age`, with the word "age" highlighted in orange. A modal dialog box titled "Define selector in:" is open, showing a list of classes: "Person", "Object", and "ProtoObject". The "Person" class is selected and underlined with a red line. There is a text input field at the bottom of the dialog and a "Cancel" button.

Débogage

The screenshot shows the Playground IDE interface. At the top, the title bar says "Playground". Below it is a toolbar with icons for "Do it", "Publish", "Bindings", "Versions", and "Pages". The main editor area shows a single line of code: "1 Person new age". Below the editor, a panel titled "Person>>age" displays the "Stack" of frames. The stack contains three frames: "Person", "UndefinedObject", and "OCReceiverDoltSemanticScope (OCDoltSemanticScope)". The "Person" frame is selected, showing its "Class" as "Person", "Method" as "age", and "Package" as "Person". The "UndefinedObject" frame shows "Dolt" as the method. The "OCReceiverDoltSemanticScope (OCDoltSemanticScope)" frame shows "evaluateDolt:" as the method. Below the stack, a toolbar contains various debugging actions: "Proceed" (highlighted with a red circle), "Into", "Over", "Through", "Run to", "Restart", "Return", "Where is?", "Create", and "Advanced Step". At the bottom, a code editor shows two lines: "1 age" and "2 ^ age". A text box on the right contains the text "Ctrl+S et Proceed".

Playground

Do it Publish Bindings Versions Pages

1 Person new age

Person>>age

Stack

Class	Method	Package
Person	age	Person
UndefinedObject	Dolt	-
OCReceiverDoltSemanticScope (OCDoltSemanticScope)	evaluateDolt:	OpalCo

Proceed Into Over Through Run to Restart Return Where is? Create Advanced Step

1 age

2 ^ age

Ctrl+S et Proceed

Débogage

The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar shows icons for file operations. The left sidebar contains a project tree with folders like 'OSWindow-Tests', 'Ombu', 'Ombu-Tests', 'OpalCompiler-Core', 'OpalCompiler-Tests', 'ParametrizedTests', 'PerformMessageCommandLi', and 'Person'. The 'Person' folder is selected. The main editor area is titled 'Person>>age'. It is divided into three panes: the left pane shows the 'Person' class with a warning icon; the middle pane shows the 'instance side' with categories 'accessing', 'initialization', and 'overrides'; the right pane shows the 'age' variable with an 'Initialize' button. A red circle highlights the 'age' variable in the right pane. Below the editor area, a toolbar contains options like 'All Packages', 'Scoped View', 'Flat', 'Hier.', 'Inst. side', 'Class side', 'Methods', 'Vars', 'Class refs.', 'Implementors', and 'Senders'. The bottom pane shows the code for the 'age' variable: `age` followed by an arrow pointing to `age`.

Conclusion

- La réflexivité est essentielle dans Pharo pour le codage en direct.
- Comprendre l'inspection, la modification et l'extension dynamique est crucial.
- Approche interactive et exploratoire du développement logiciel.

Exercices

- Créez une classe **Person** et ajoutez dynamiquement les méthodes *hi* et *bye* qui retournent les chaînes de caractères correspondantes.
- Dans la classe **Person** créez une méthode *goodMood* qui génère aléatoirement un nombre entre 1 et 10. Si ce nombre est supérieur à 5, modifiez le code de la méthode *hi* de manière programmatique : elle doit maintenant retourner la chaîne de caractères **'Hello sunshine!'**.

Exercices

Ajoutez à la classe `Person` la variable d'instance *hobby*.

Ajoutez à la classe `Person` de manière programmatique:

la méthode *initialize*, qui ajoute à la collection *hobby* 2 éléments aléatoires parmi 'lecture', 'programmation', 'sortir', 'sport', 'cuisine'.

la méthode *talkTo: aPerson*, qui vérifie quels sont les hobbies de *aPerson*, génère dynamiquement les méthodes pour parler de ces hobbies, par exemple *talkAboutSport: aPerson*, qui retourne une phrase associée, comme "Le football, c'est cool".

Exécutez ces méthodes (*talkAbout...*) directement dans la méthode *talkTo*.

Supprimer programmatiquement toutes les méthodes dont les sélecteurs commencent par 'talkAbout' de la classe `Person`.

Exercices

Créer une méthode avec les fonctionnalités suivantes :

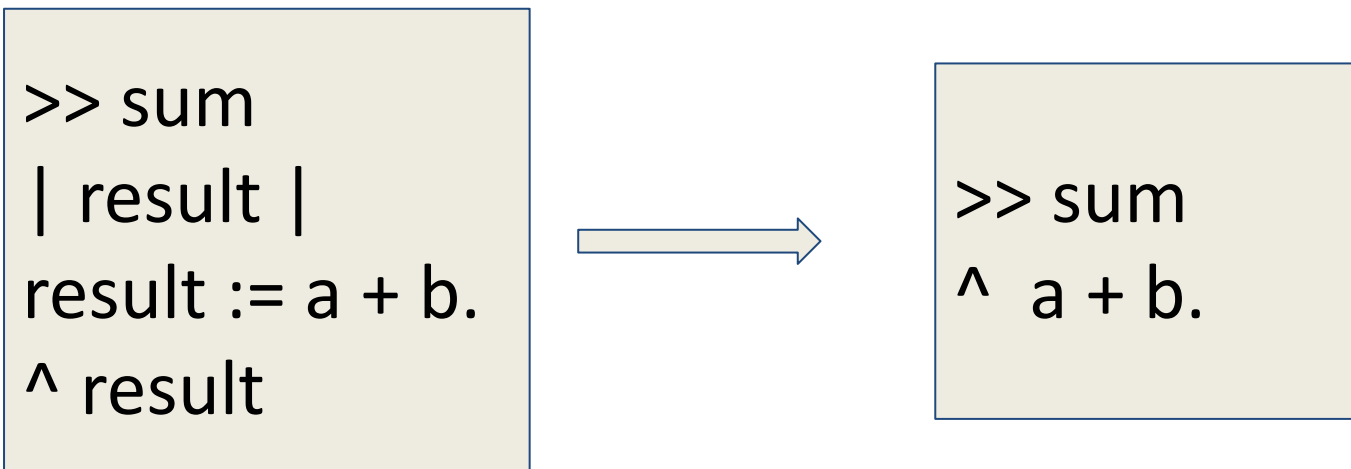
1. Elle crée un package nommé **MetaExercise** de manière programmatique.
2. Dans ce package, elle crée une classe portant le même nom (**MetaExercise**).
3. Dans cette classe, elle crée une méthode `add:a to:b` qui retourne la somme de `a` et `b`.
4. Elle exécute la méthode `add:a to:b` et retourne son résultat.
5. Elle effectue un test confirmant que le résultat retourné est correct.

Pensez à vérifier si un package portant ce nom existe déjà et à le supprimer du système avant de le créer.

Exercices: Refactoring Inline Temp

Implémentez un visitor qui réalise le refactoring Inline Temp:

supprimer une variable temporaire utilisée uniquement pour retourner une valeur et la remplacer par son expression.



[Exercices]

- Ajoutez la prise en charge des LLM avec <https://github.com/Evref-BL/Pharo-LLMAPI>
- Modifiez la méthode *talkTo: aPerson* afin que, sur la base des hobbies, elle génère une requête vers le LLM demandant quoi poser comme question sur le hobby d'une personne. Recevez la réponse du LLM, modifiez la méthode *talkAbout...* en conséquence, puis exécutez la méthode *talkAbout*.
- Écrivez le résultat dans le transcript ou dans un fichier (avec [FileStream](#)).

[Exercices]

Créez deux méthodes dans la classe **Person** :

- **talkTo: aPerson tour: aCounter**
- **talkAboutHobby: aQuestion with: aPerson tour: aCounter**

Définition et comportement :

1. **talkTo: aPerson tour: aCounter**
 - Analyse les hobbies de aPerson.
 - Génère une question sur l'un des hobbies en utilisant un LLM.
 - Demande au LLM de répondre en une phrase.
 - Appelle **talkAboutHobby:with:tour:** avec la réponse du LLM comme **aQuestion** et **aCounter** comme nombre de tours de conversation.
2. **talkAboutHobby: aQuestion with: aPerson tour: aCounter**
 - Envoie **aQuestion** au LLM.
 - Demande une réponse en une phrase et une nouvelle question en une phrase.
 - Appelle **talkAboutHobby:with:tour:** avec aPerson comme récepteur, self comme premier argument, la réponse du LLM comme aQuestion, et aCounter - 1.
 - Si aCounter = 0, ne pas rappeler la méthode et terminer l'exécution.

Écrivez le dialogue dans le transcript ou dans un fichier (avec **FileStream**).